Rethinking High-Level Synthesis Design Space Exploration from a Contrastive Perspective

Huiliang Hong, Chenglong Xiao*,Shanshan Wang

Department of Computer Science, Shantou University, China
*Corresponding author: Chenglong Xiao
E-mail:{22hlhong,chlxiao,sswang}@stu.edu.cn

2024 IEEE 42nd International Conference on Computer Design (ICCD) | 979-8-3503-8040-8/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICCD63220.2024.00035

***Abstract*—In High-Level Synthesis (HLS), the design space of RTL implementations growing exponentially with the number of directives, coupled with the expensive nature of the synthesis process to obtain the characteristics of a single RTL implementation, pose challenges in exhaustively exploring the design space to identify Pareto-optimal designs. To find Pareto-optimal designs, most existing studies adopt regression methods or call HLS tools to provide the performance and cost of each design. Considering the essence of finding Pareto-optimal designs, we observed that predicting the relative dominance relationship between designs is sufficient while predicting the absolute performance and cost of each design is not a must. We propose an alternative approach to identify Pareto-optimal designs by establishing a Contrastive Learning (CL) framework that determines the dominance relationship between designs based on classification methods. Furthermore, we integrate the CL framework with three state-of-the-art Design Space Exploration (DSE) approaches, namely CL-DSE, with the goal of effectively reducing the number of syntheses. Experimental comparisons of 39 regression methods and 17 CL-assisted classification methods reveal that the classification methods generally overwhelm the regression methods for predicting the dominance relationship between designs. Experimental results also reveal that the CL-DSE approaches can significantly decrease the overall running time by reducing the number of syntheses, while retaining the quality of results in comparison to the DSE approaches. The proposed CL-DSE approaches are publicly available at https://github.com/hong64/CL-DSE.**

***Index Terms*—High-Level Synthesis, Design Space Exploration, Online Contrastive Learning, Classification**

I. Introduction

High-Level Synthesis (HLS) empowers designers to architect Register-Transfer Level implementations, characterized by performance and cost, at a high-level. In the context of HLS, several functionally identical variants of the same design can be generated without requiring changes to the behavioral description by using synthesis directives in the form of pragmas that are tagged in the source code as comments. This makes it possible for designers to conduct a design space exploration (DSE) of the design and subsequently choose the microarchitecture that best suits the requirements. Thus, there are numerous ways to implement a design at RTL with different trade-offs in terms of performance, cost, and power consumption, among other metrics, leading the design space to be large-scale. Compounding the issue, the synthesis process for obtaining the characteristics of one design is time-consuming, necessitating minutes to hours by invoking the HLS tool [1]. Consequently, HLS requires effective design space exploration approaches to navigate the extensive design space.

DSE approaches traverse the design space to discover Pareto-optimal designs that strike the best balance between conflicting objectives, such as performance and cost. To achieve efficient DSE, several state-of-the-art heuristic-based approaches, such as Cluster-based [2], Lattice-based [3], and Genetic Algorithm [4], are utilized to navigate the design space. These approaches may require a large number of syntheses, leading to extended timeframes for completing the DSE task, which are further exacerbated when the synthesis time is prolonged. To find the Pareto-optimal designs, the essence is how to efficiently and accurately compare the designs. Machine learning (ML) techniques that mainly work in a regression way are widely adopted [5]–[7] to provide the estimation of performance and cost of each design to reduce the time for executing the tedious HLS process. In this paper, we rethink HLS DSE from a constrative perspective by proposing an online contrastive learning framework for predicting the dominance relationship between designs. Our work offers the following contributions:

- A contrastive learning framework based on a binary classification is introduced to determine the dominance relationship between designs through pairwise comparison. This approach simplifies the prediction task and enhances the robustness of the model by inherently increasing the amount of available training data.
- Extensive experiments have been conducted to compare the accuracy of 39 regression methods and 17 classification methods for predicting the dominance relationship between designs. The results demonstrate that classification methods generally outperform regression methods.
- We integrate the Contrastive Learning (CL) framework with state-of-the-art DSE approaches. Experimental results show that CL-DSE can reduce a significant amount

This work was supported in part by Scientific Research Project of Colleges and Universities in Guangdong Province (2021ZDZX1027), Guangdong Basic and Applied Basic Research Foundation (2022A1515110712 and 2023A1515010077).

2576-6996/24/$31.00 ©2024 IEEE
DOI 10.1109/ICCD63220.2024.00035

Authorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on January 07,2025 at 04:04:01 UTC from IEEE Xplore. Restrictions apply.

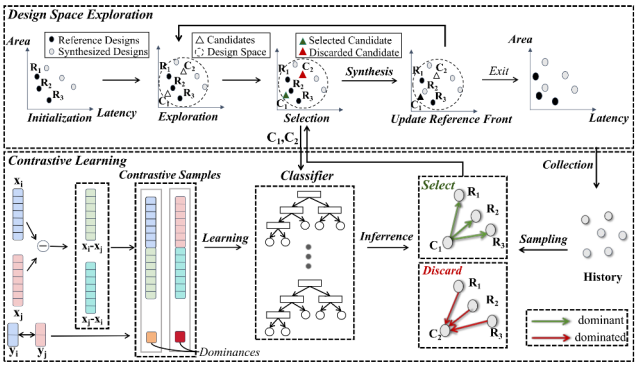


Fig. 1. The overview of the proposed CL-DSE framework.

of synthesis, while maintaining the quality of results compared to the original DSE approaches.

II. Proposed Method

A. Overview

The overall CL-DSE framework, depicted in Fig.1, consists of two components: Design Space Exploration and Contrastive Learning (CL). In the CL component, we train a classifier using historical synthesized paired designs collected during the DSE process, enabling it to comprehend the contrastive differences among designs. The DSE component's objective is to steer the search process towards the promising area in the design space, generating various progressive candidate designs for CL component identification.

The whole CL-DSE approach consists of four main procedures: Initialization, Exploration, CL-assisted Selection and Reference Front Update. The initialization phase establishes an initial reference front. The exploration phase discovers new candidates for synthesis. The CL-assisted Selection employs the embedded CL component to identify the pros and cons of the candidates. Candidates inferred as superior are synthesized using the HLS tool, with results updating the reference front and being collected in a historical archive for future training. The final reference front will be output until the DSE exit condition is meet. The CL-assisted Selection can be summarized in three parts: Comparison, Learning, Inference, whose particular realization is described as follows.

B. Comparison

The comparison process generates a contrastive dataset from the historical archive D for model training, which is represented as:

$$D_{cl} = \{(x_i \,||\, (x_i - x_j), dom_{ij}) | x_i, x_j \in D\} \tag{1}$$

where the contrastive samples are consist of two aspects: contrastive feature and dominance label. The contrastive feature $x_i||(x_i - x_j)$ is established by sequentially concatenating the configuration vector x_i and the differential vector $(x_i - x_j)$. The actual dominance label dom_{ij} is determined by comparing characteristics (y_i and y_j) of two designs as:

$$dom_{ij} = \begin{cases} 0, & \textbf{if } y_i \succeq y_j \\ 1, & \textbf{else} \end{cases} \tag{2}$$

$$y_i \succeq y_j \Leftrightarrow A_i \geqslant A_j \, and \, L_i \geqslant L_j \tag{3}$$

$dom_{ij} = 0$ indicates that design d_i is "dominated", meaning it has no advantage over design d_j in terms of both area and latency. Conversely, $dom_{ij} = 1$ indicates a "dominant" relation, signifying that design d_i has an advantage over design d_j at least one of the multiple objectives. By comparing each pairs in D, the available volume of D_{cl} for binary classification task inherently doubles compared to the original dataset.

C. Learning

In this paper, we train one classifier based on the contrastive samples in D_{cl}. Since the classifier is optional, seventeen classifier's performance are evaluated in the extensive experiments to choose one appropriate classifier, where the XGBoost classifier is finally selected. The XGBoost classifier integrates K weak learners with lower classification accuracy, optimizing them sequentially in a boosting manner to generate a strong model with high accuracy. Given one contrastive feature $x_i||(x_i - x_j)$ as input, the output of the model can be represented as:

$$[prob, \hat{dom}]_{ij} = clf(x_i||(x_i - x_j)) \tag{4}$$

where $prob$ represent the probability of its predicting dominance label $\hat{dom}_{ij}$. We select the cross entropy (CE) loss function to quantify the difference between the predicted dominance label $\hat{dom}_{ij}$ and the actual dominance label dom_{ij} to guide the classifier training.

D. Inference

The inference phase aims to infer the dominance relations between the candidate c and the current reference front designs r_i, further select the predicted promising candidates for HLS synthesis. For each pair $< c, r_i >$, the CL framework infer their dominance relation $\hat{dom}_{cr_i}$ based on the trained classifier. $\hat{dom}_{cr_i} = 0$ indicates that the candidate is dominated by the reference design. In contrast, $\hat{dom}_{cr_i} = 1$ signifies the candidate is dominant. Summarize each inference on the reference design r_i and candidate c , the correlation between the candidate and the reference front can be divided into two scenarios:

- the candidate is dominant over all reference designs.
- the candidate only partially dominates over the reference designs or may not dominate any reference designs at all.

Candidates in the first scenario are considered more promising to improve the reference front update and are therefore selected for HLS synthesis. Their synthesis results will be collected in the history archive for use in subsequent training iterations. Candidates in the second scenario will be discarded and marked so that the DSE approach can disregard them in future iterations.

Authorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on January 07,2025 at 04:04:01 UTC from IEEE Xplore. Restrictions apply.

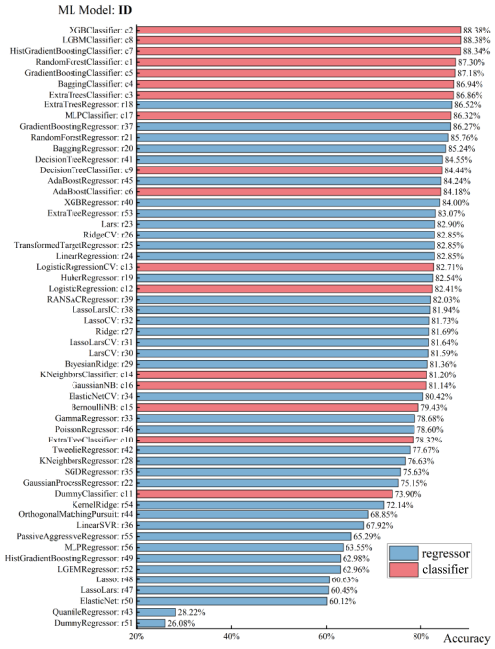


Fig. 2. Prediction accuracy of dominance relationship achieved by 39 regressors and 17 CL-based classifiers.

III. Experimental Results

A. Experiments Setup

The benchmarks for experiments are derived from DB4HLS [8], which is a publicly available HLS database based on the benchmark functions in Machsuite, providing well-defined design spaces and related synthesis outcomes. We integrated the proposed contrastive learning framework with three state-of-the-art DSE methods: Lattice-Traversing [3] and Cluster-Based [2] and ϵ-GA [4], which are labeled as CL-Cluster, CL-Lattice and CL-ϵ-GA respectively. Lattice-Traversing, Cluster-Based and ϵ-GA are three effective synthesis-driven DSE approaches, which are advantageous in accuracy, transfer and implementation [9]. We run the source codes of Lattice-Traversing and Cluster-Based from [3] and re-implemented the ϵ-GA according to pseudo-code in [4]; setting the radius hyper-parameter of Lattice-Traversing to 1 and keep the Cluster-Based's hyper-parameters unchanged. Furthermore, the CL-DSE (CL-Lattice, CL-Cluster, CL-ϵ-GA) and DSE (Lattice-Traversing, Cluster-Based, ϵ-GA) use identical initial samples to further exclude the random effects.

B. Regression vs Classification

We first conduct an extensive experiments on comparing the prediction accuracy of dominance relationship achieved by 39 regressors and 17 classifiers which are available from the scikit-learn toolbox. Experiments are carried out on the selected benchmark functions, specifically those with an associated design space size surpassing 100. For classification methods, we adopt the proposed contrastive learning component for training and inference. The hyper-parameters for each model are retained at their default settings, as specified in the scikit-learn toolbox. For each benchmark, we repeat the evaluation of each model 20 times to obtain its average performance. In each evaluation, $|DS| \times 5\%$ samples are randomly selected for the 5-fold cross-validation.

In Fig. 2, the y-axis reports the ML models' names and their IDs. The results are listed in the order of the average accuracy over all benchmarks. We can make the following observations: ① Among the 17 classifiers, the highest achieved accuracy surpasses the accuracy of the most proficient regressor (ExtraTreesRegressor), identified from a pool of 39 regressors, by a margin of 1.86%. ② The number of CL classifiers (=8) listed in the top ten models far exceeds the number of regressors (=2), indicating the robustness of the CL classifiers. ③ Ensemble learning techniques achieve higher performance across both CL classifiers (c8, c2, c7, c5, c1, c4, c3) and regressors (r18, r4).

C. Evaluation of the proposed CL-DSE approaches

In the experiments, we compare the CL-DSE and DSE approaches on two evaluation metrics: Average Distance from the Referenced Set (ADRS), the DSE time (T_{DSE}). The ADRS metric is used for measuring the quality of the results (QoRs), which can be calculated as:

$$ADRS(P,R) = \frac{1}{|P|} \cdot \sum_{p_i \in P} \min_{r_i \in R} (Distance(p_i, r_i))$$
$$Distance(p_i, r_i) = \max(|\frac{A(r_i) - A(p_i)}{A(p_i)}|, |\frac{L(r_i) - L(p_i)}{L(p_i)}|) \quad (5)$$

where $A(\cdot)$ quantify one design's resource consumption as:

$$Area_i = \frac{1}{4} * (\frac{FF_i}{FF_{avail}} + \frac{LUT_i}{LUT_{avail}} + \frac{BRAM_i}{BRAM_{avail}} + \frac{DSP_i}{DSP_{avail}}) \quad (6)$$

$L(\cdot)$ is the average latency to measure one design's performance. The DSE time is coarsely calculated as:

$$T_{DSE} = runtime + T_{hls} \quad (7)$$

where $runtime$ represents the time for running the DSE approach (excluding the time for executing the synthesis process) and T_{hls} represents the time for synthesizing designs.

DSE Time. We compare the overall DSE time of CL-DSE approaches and DSE approaches (Table I). The Improv. columns report the improvement achieved by CL-DSE approaches over DSE approaches on DSE time. We can make the following observations: ① The integration of the CL component can greatly reduce the synthesis time; respectively reducing 30.03, 9.25, and 0.52 hours in Lattice-Traversing, ϵ-GA and Cluster-Based on average. ② The DSE time heavily depends on the synthesis time. With greatly reducing the synthesis time, the DSE time of the Lattice-Traversing, ϵ-GA, and Cluster-Based approach can be accelerated by 5.3x, 2.4x, and 1.1x respectively. ③ The CL-DSE's runtime is sometimes increased due to the the model training and prediction requires for the extra time. However, this extra time is not significant.

ADRS. Table II presents a comparison of the average ADRS values achieved by both the CL-DSE and the original DSE approaches on each benchmark. It should be noted that the dominance prediction model in CL-DSE is trained with the

Authorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on January 07,2025 at 04:04:01 UTC from IEEE Xplore. Restrictions apply.

TABLE I
DSE TIME COMPARISON OF DIFFERENT CL-DSE APPROACHES AND DSE APPROACHES. THE IMPROV. COLUMNS PRESENT THE IMPROVEMENT ON THE RUNTIME/SYNTHESIS TIME AFTER INTEGRATING THE CONTRASTIVE LEARNING COMPONENT

DSE Time (in hours)												
Benchmark	**CL-Lattice**	**Lattice-Traversing [3]**	**Improv.**	**SpeedUp**	**CL-ϵ-GA**	**ϵ-GA [4]**	**Improv.**	**SpeedUp**	**CL-Cluster**	**Cluster-Based [2]**	**Improv.**	**SpeedUp**
spmv ellpack	2.6	21.9	0/19.3	8.4x	2.1	4.6	0/2.5	2.2x	2.5	3.1	0/0.6	1.2x
bfs bulk	7.8	26.4	0/18.6	3.4x	8.4	19.7	0.1/11.2	2.3x	7.2	7.3	0.1/0	1.0x
md knn	11.5	82.1	-0.1/70.7	7.1x	7.7	19.6	0/11.9	2.5x	5.7	5.9	0/0.2	1.0x
viterbi	5.1	9.7	0.1/4.5	1.9x	5.8	17.1	0.1/11.2	2.9x	4.1	4.3	0.1/0.1	1.0x
gemm	7	43.8	0/36.8	6.3x	7	21.7	0/14.7	3.1x	6.2	6.7	0/0.5	1.1x
fft	1.9	19.8	-0.1/18	10.4x	4.2	10.4	0/6.2	2.5x	1.9	1.9	-0.1/0.1	1.0x
sort merge	10.7	61.3	0.1/50.5	5.7x	5.8	11.3	0/5.5	1.9x	8.5	10.1	0.1/1.5	1.2x
stencil	10	29.7	0/19.7	3.0x	8.4	24.1	0/15.7	2.9x	7.3	8.3	-0.1/1.1	1.1x
radix sort	11.8	58.2	0/46.4	4.9x	8.8	20.9	0/12.1	2.4x	9	9.6	0/0.6	1.1x
aes	0.7	1.6	0/0.9	2.3x	0.7	1.8	0/1.1	2.6x	0.6	0.7	0/0.1	1.2x
backprop	12	56.9	0/44.9	4.7x	17.6	27.2	-0.1/9.7	1.5x	19.3	21.7	1.5/0.9	1.1x
AVERAGE	**7.37**	**37.4**	**0/30.03**	**5.3x**	**6.95**	**16.22**	**0.02/9.25**	**2.4x**	**6.57**	**7.24**	**0.15/0.52**	**1.1x**

TABLE II
ADRS RESULTS OF DIFFERENT CL-DSE APPROACHES AND DSE APPROACHES.

ADRS									
Benchmark	**CL-Lattice**	**Lattice-Traversing [3]**	**Δ**	**CL-ϵ-GA**	**ϵ-GA [4]**	**Δ**	**CL-Cluster**	**Cluster-Based [2]**	**Δ**
spmv ellpack	0.026	0	0.026	0.067	0.065	0.002	0.122	0.105	0.017
bfs bulk	0.005	0	0.005	0.021	0.016	0.005	0.015	0.013	0.002
md knn	0.011	0	0.011	0.005	0.011	-0.006	0.058	0.065	-0.007
viterbi	0	0	0	0.001	0.001	0	0.002	0.002	0
gemm	0.049	0.023	0.026	2.527	3.715	-1.188	0.103	0.094	0.009
fft	0.002	0	0.002	0.008	0.009	-0.001	0.002	0.002	0
sort merge	0	0	0	0.003	0.004	-0.001	0.026	0.005	0.021
stencil	0.119	0.014	0.105	0.062	0.061	0.001	0.472	0.465	0.007
radix sort	0.219	0.013	0.206	0.357	1.599	-1.242	1.073	0.902	0.171
aes	0.094	0.001	0.093	0.037	0.028	0.009	1.317	1.278	0.039
backprop	0.008	0.002	0.006	0.026	0.029	-0.003	0.041	0.03	0.011
AVERAGE			0.044			-0.22			0.025

latency and area results generated by the HLS tool, and the DSE approaches directly use the results from the HLS tool. Moreover, we simply integrate the proposed CL framework into the DSE approaches and do not change the core procedures of DSE approaches. Hence, it is not surprising to see that the CL-DSE approaches generally achieve slightly inferior performance in terms of QoRs compared to the original DSE approaches. From the results, we can observe that the ADRS difference is maintained at less than 0.04 for most of the benchmarks. Subsequently, this relatively small deviation suggests that the QoRs achieved by the CL-DSE approaches are close to those obtained by the original DSE approaches. It is also interesting to observe that we may get $\Delta < 0$ for $\epsilon - GA$. The possible reason is that, with the prediction model in the CL framework, some inferior candidate designs may be selected. For genetic algorithms, involving some inferior solutions can increase the diversity of the population and sometimes lead to better results.

IV. CONCLUSION

In this paper, we revisit HLS DSE from a contrastive perspective by introducing an online contrastive learning framework that generates a predictive model for determining the dominance relationship between designs. Extensive experiments have been performed to compare the accuracy of regression models and classification models. Furthermore, the proposed framework can be flexibly integrated into the state-of-the-art DSE approaches. We believe that this work may inspire more work on dealing with HLS DSE from a contrastive perspective.

REFERENCES

[1] Hanchen Ye, Cong Hao, Jianyi Cheng, Hyunmin Jeong, Jack Huang, Stephen Neuendorffer, and Deming Chen. Scalehls: A new scalable high-level synthesis framework on multi-level intermediate representation. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 741–755. IEEE, 2022.

[2] Lorenzo Ferretti, Giovanni Ansaloni, and Laura Pozzi. Cluster-based heuristic for high level synthesis design space exploration. *IEEE Transactions on Emerging Topics in Computing*, 9(1):35–43, 2018.

[3] Lorenzo Ferretti, Giovanni Ansaloni, and Laura Pozzi. Lattice-traversing design space exploration for high level synthesis. In *2018 IEEE 36th International Conference on Computer Design (ICCD)*, pages 210–217. IEEE, 2018.

[4] Yiheng Gao and Benjamin Carrion Schafer. Effective high-level synthesis design space exploration through a novel cost function formulation. In *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*, pages 1–5. IEEE, 2021.

[5] Sayandip De, Muhammad Shafique, and Henk Corporaal. Delay prediction for asic hls: comparing graph-based and nongraph-based learning models. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 42(4):1133–1146, 2022.

[6] Nan Wu, Yuan Xie, and Cong Hao. Ironman-pro: Multiobjective design space exploration in hls via reinforcement learning and graph neural network-based modeling. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 42(3):900–913, 2022.

[7] Nan Wu, Yuan Xie, and Cong Hao. Ironman: Gnn-assisted design space exploration in high-level synthesis via reinforcement learning. In *Proceedings of the 2021 on Great Lakes Symposium on VLSI*, pages 39–44, 2021.

[8] Lorenzo Ferretti, Jihye Kwon, Giovanni Ansaloni, Giuseppe Di Guglielmo, Luca Carloni, and Laura Pozzi. Db4hls: a database of high-level synthesis design space explorations. *IEEE Embedded Systems Letters*, 13(4):194–197, 2021.

[9] Benjamin Carrion Schafer and Zi Wang. High-level synthesis design space exploration: Past, present, and future. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 39(10):2628–2639, 2019.

Authorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on January 07,2025 at 04:04:01 UTC from IEEE Xplore. Restrictions apply.